NORMALIZATION.md

Job Portal with Intelligent Filtering — Milestone 2

Formal Normalization Walkthrough (1NF → 2NF → 3NF)

Prepared By: Aleena Rafiq & Talha Muhammad Bangash
Submitted To: Sir Ali Hassan
Version: 1.0 — Normalization & Schema Refinement
Date: May 2026

Table of Contents

1. Overview & Methodology
2. Pre-Normalization: Conceptual Flat Table
3. First Normal Form (1NF)
4. Second Normal Form (2NF)
5. Third Normal Form (3NF)
6. Schema Refinements & Redundancy Analysis
7. Final Normalized Schema
8. Normalization Summary Table

---

1. Overview & Methodology

What is Normalization?

Normalization is the systematic process of organizing a relational database schema to:

- **Eliminate data redundancy** — the same data should not be stored in multiple places.
- **Eliminate update anomalies** — changing one fact should require updating exactly one row.

- **Eliminate insertion anomalies** — it should be possible to add one entity without requiring another.
- **Eliminate deletion anomalies** — deleting one entity should not inadvertently destroy unrelated data.

Normal Forms Applied

Normal Form	Rule
1NF	All attributes are atomic (single-valued). No repeating groups. Every row is uniquely identifiable.
2NF	Satisfies 1NF. Every non-key attribute is **fully functionally dependent** on the entire primary key (eliminates partial dependencies — only relevant for composite PKs).
3NF	Satisfies 2NF. Every non-key attribute is **directly dependent on the primary key only** — no transitive dependencies through another non-key attribute.

Notation Used

```
PK  = Primary Key
FK  = Foreign Key
FD  = Functional Dependency  (A → B means "A determines B")
PD  = Partial Dependency     (part of composite PK → non-key attribute)
TD  = Transitive Dependency  (non-key A → non-key B)
✓   = Already satisfies this normal form; no change required
✗   = Violation found; restructuring applied
```

---

2. Pre-Normalization: Conceptual Flat Table

Before decomposition, imagine all recruitment data stored in a single flat record. This illustrates every violation the normalization process must resolve.

```
RECRUITMENT_FLAT (
  UserID, UserName, UserEmail, UserPasswordHash, UserRole, UserCreatedAt,
  CompanyID, CompanyName, CompanyIndustry, CompanyLocation, CompanyContactEmail,
  JobID, JobTitle, JobDescription, JobLocation, JobSalary, JobDeadline, JobStatus
  SkillID_1, SkillName_1, SkillCategory_1, RequirementLevel_1,
  SkillID_2, SkillName_2, SkillCategory_2, RequirementLevel_2,   ← repeating grou
```

```
  ApplicantID, ResumeURL, ExperienceYears, Bio, Gender, University,
  AppSkillID_1, AppSkillName_1, ProficiencyLevel_1,
  AppID, AppliedDate, AppStatus,
  InterviewID, ScheduledDate, Mode, Result
)
```

Violations visible immediately:

- Repeating skill groups (SkillID_1, SkillID_2 ...) violate **1NF**.
- Skill name and category depend only on SkillID, not the full application key — **partial dependency violating 2NF**.
- Company attributes repeat on every job row — **redundancy and update anomaly**.
- Interview data repeats applicant information — **transitive dependency violating 3NF**.

The following sections walk through each table systematically.

---

3. First Normal Form (1NF)

> **Rule:** All column values must be atomic (indivisible). No repeating groups or multi-valued attributes. Each row must be uniquely identifiable by a primary key.

3.1 USERS

Proposed Attributes: `UserID, Name, Email, PasswordHash, Role, CreatedAt`

1NF Check:

Attribute	Atomic?	Issue
UserID	✓	Single integer
Name	✓	Single string (full name treated as one unit)
Email	✓	Single string
PasswordHash	✓	Single hash string
Role	✓	Single ENUM value
CreatedAt	✓	Single DATETIME value

Repeating Groups? None.
Unique Row Identifier? `UserID` (auto-incremented PK).

> ✓ **USERS already satisfies 1NF.** All attributes are atomic, no multi-valued fields exist, and every row is uniquely identified by `UserID`.

3.2 COMPANIES

Proposed Attributes: `CompanyID, Name, Industry, Location, ContactEmail, UserID`

1NF Check:

Attribute	Atomic?	Issue
CompanyID	✓	Single integer
Name	✓	Single string
Industry	✓	Single string
Location	✓	Single string
ContactEmail	✓	Single string
UserID	✓	Single FK integer

Repeating Groups? None.
Unique Row Identifier? `CompanyID`.

> ✓ **COMPANIES already satisfies 1NF.** All values are atomic, no arrays or lists exist, and `CompanyID` uniquely identifies each row.

3.3 JOBS

Proposed Attributes: `JobID, Title, Description, CompanyID, Location, Salary, Deadline, Status`

1NF Check:

Attribute	Atomic?	Issue
JobID	✓	Single integer

Title	✓	Single string
Description	✓	TEXT is one logical value
CompanyID	✓	Single FK
Location	✓	Single string
Salary	✓	Single DECIMAL
Deadline	✓	Single DATE
Status	✓	Single ENUM

Repeating Groups? A naive design might store required skills as `Skills: "Python, SQL, Java"` — a comma-separated multi-value in one column. This is **rejected**; skills are instead stored in the separate `JOB_SKILLS` junction table.

> ✓ **JOBS already satisfies 1NF.** All attributes are atomic. Multi-valued skill data is correctly externalized to JOB_SKILLS, not stored as a delimited list.

3.4 APPLICANTS

Proposed Attributes: `ApplicantID, UserID, ResumeURL, ExperienceYears, Bio, Gender, University`

1NF Check:

Attribute	Atomic?	Issue
ApplicantID	✓	Single integer
UserID	✓	Single FK
ResumeURL	✓	Single URL string
ExperienceYears	✓	Single integer
Bio	✓	TEXT is one logical value
Gender	✓	Single ENUM
University	✓	Single string

Repeating Groups? A naive design might store multiple skills in a `Skills` column as `"Python:Expert, SQL:Intermediate"`. This is **rejected**; skills are externalized to `APPLICANT_SKILLS`.

> ✓ **APPLICANTS already satisfies 1NF.** All attributes are atomic. Multi-valued skill data is correctly externalized to APPLICANT_SKILLS.

3.5 SKILLS

Proposed Attributes: `SkillID, SkillName, Category`

1NF Check:

Attribute	Atomic?	Issue
SkillID	✓	Single integer
SkillName	✓	Single string
Category	✓	Single string

> ✓ **SKILLS already satisfies 1NF.** Three simple atomic attributes; `SkillID` uniquely identifies each row.

3.6 JOB_SKILLS

Proposed Attributes: `JobSkillID, JobID, SkillID, RequirementLevel`

1NF Check:

Attribute	Atomic?	Issue
JobSkillID	✓	Single integer surrogate PK
JobID	✓	Single FK
SkillID	✓	Single FK
RequirementLevel	✓	Single ENUM

Repeating Groups? This table *is itself* the solution to what would otherwise be a repeating group (multiple skills per job). Each row stores exactly one job–skill pair.

> ✓ **JOB_SKILLS already satisfies 1NF.** It was created specifically to resolve the repeating skill group that would exist if skills were stored inside the JOBS table.

3.7 APPLICANT_SKILLS

Proposed Attributes: `AppSkillID, ApplicantID, SkillID, ProficiencyLevel`

1NF Check: Identical reasoning to JOB_SKILLS — each row represents exactly one applicant–skill pair with a single proficiency value.

> ✓ **APPLICANT_SKILLS already satisfies 1NF.** Created to resolve repeating skill groups that would exist if skills were stored inside APPLICANTS.

3.8 APPLICATIONS

Proposed Attributes: `AppID, ApplicantID, JobID, AppliedDate, Status`

1NF Check:

Attribute	Atomic?	Issue
AppID	✓	Single integer
ApplicantID	✓	Single FK
JobID	✓	Single FK
AppliedDate	✓	Single DATE
Status	✓	Single ENUM

> ✓ **APPLICATIONS already satisfies 1NF.** All attributes are atomic and `AppID` uniquely identifies each application record.

3.9 INTERVIEWS

Proposed Attributes: `InterviewID, AppID, ScheduledDate, Mode, Result`

1NF Check:

Attribute	Atomic?	Issue

InterviewID	✓	Single integer
AppID	✓	Single FK
ScheduledDate	✓	Single DATETIME
Mode	✓	Single ENUM
Result	✓	Single ENUM

> ✓ **INTERVIEWS already satisfies 1NF.** All attributes are atomic.

1NF Summary

> **All 9 tables satisfy 1NF as designed.** The schema avoided the most common 1NF violation — storing multi-valued skill data as comma-separated strings — by proactively creating the JOB_SKILLS and APPLICANT_SKILLS junction tables. Every table has a single-column atomic primary key.

---

4. Second Normal Form (2NF)

> **Rule:** Must satisfy 1NF. Every non-key attribute must be **fully functionally dependent** on the **entire** primary key. Partial dependencies (where a non-key attribute depends on only *part* of a composite PK) must be eliminated.
>
> **Note:** 2NF violations are only possible when a table has a **composite primary key**. Tables with a single-column PK automatically satisfy 2NF.

4.1 USERS — Single PK: `UserID`

No composite key → **automatically satisfies 2NF**.

> ✓ **No partial dependencies possible.** All attributes depend on `UserID`.

4.2 COMPANIES — Single PK: `CompanyID`

No composite key → **automatically satisfies 2NF**.

> ✓ **No partial dependencies possible.** All attributes depend on `CompanyID`.

4.3 JOBS — Single PK: `JobID`

No composite key → **automatically satisfies 2NF**.

> ✓ **No partial dependencies possible.** All attributes depend on `JobID`.

4.4 APPLICANTS — Single PK: `ApplicantID`

No composite key → **automatically satisfies 2NF**.

> ✓ **No partial dependencies possible.** All attributes depend on `ApplicantID`.

4.5 SKILLS — Single PK: `SkillID`

No composite key → **automatically satisfies 2NF**.

> ✓ **No partial dependencies possible.** `SkillName` and `Category` both depend solely on `SkillID`.

4.6 JOB_SKILLS — Composite Natural Key: (`JobID` , `SkillID`)

This table uses a surrogate PK `JobSkillID`, but the **natural key** is the composite pair `(JobID, SkillID)`. The 2NF analysis is performed against the natural key to detect logical partial dependencies.

Functional Dependencies:

```
(JobID, SkillID)  →  RequirementLevel   ✓ FULL dependency
                                           (the requirement level for Python in J
                                            is specific to that job-skill combina

JobID alone       →  Title, CompanyID   ✗ But these are NOT attributes of this ta
SkillID alone     →  SkillName          ✗ But this is NOT an attribute of this ta
```

Assessment: `RequirementLevel` is the only non-key attribute, and it is fully determined by the entire `(JobID, SkillID)` pair — not by either key part alone. The job title and skill name live in their respective parent tables (JOBS, SKILLS), not here.

> ✓ **JOB_SKILLS satisfies 2NF.** `RequirementLevel` is fully functionally dependent on the composite natural key `(JobID, SkillID)`. No partial dependencies exist within this

> table.

4.7 APPLICANT_SKILLS — Composite Natural Key: (`ApplicantID` , `SkillID`)

Functional Dependencies:

```
(ApplicantID, SkillID)  →  ProficiencyLevel   ✓ FULL dependency
                                                 (an applicant's proficiency in
                                                  is specific to that applicant-

ApplicantID alone       →  ResumeURL, Bio     ✗ Not attributes of this table
SkillID alone           →  SkillName          ✗ Not an attribute of this table
```

Assessment: `ProficiencyLevel` is the only non-key attribute and is fully determined by the `(ApplicantID, SkillID)` pair.

> ✓ **APPLICANT_SKILLS satisfies 2NF.** `ProficiencyLevel` is fully functionally dependent on the composite natural key. No partial dependencies exist.

4.8 APPLICATIONS — Composite Natural Key: (`ApplicantID` , `JobID`)

This table uses a surrogate PK `AppID` , but the natural unique key is `(ApplicantID, JobID)` . Analyzing against this:

Functional Dependencies:

```
(ApplicantID, JobID)  →  AppliedDate    ✓ FULL dependency
                                           (the application date is specific to t
                                            applicant-job pair)

(ApplicantID, JobID)  →  Status         ✓ FULL dependency
                                           (the pipeline status belongs to this a

ApplicantID alone     →  ResumeURL      ✗ Not an attribute of this table
JobID alone           →  Title          ✗ Not an attribute of this table
```

> ✓ **APPLICATIONS satisfies 2NF.** Both `AppliedDate` and `Status` are fully dependent on the `(ApplicantID, JobID)` natural key. No applicant-only or job-only attributes are stored in this table.

4.9 INTERVIEWS — Single PK: `InterviewID`

No composite key → **automatically satisfies 2NF**.

> ✓ **No partial dependencies possible.** All attributes depend on `InterviewID`.

2NF Summary

> **All 9 tables satisfy 2NF.** The two junction tables (JOB_SKILLS, APPLICANT_SKILLS) and the transactional table (APPLICATIONS) were the only candidates for partial dependency violations. In each case, the non-key attributes (`RequirementLevel`, `ProficiencyLevel`, `AppliedDate`, `Status`) are fully determined by the entire composite natural key — not by either part alone. This is because all parent-entity attributes (job titles, skill names, applicant bios) were correctly placed in their own parent tables during schema design.

---

5. Third Normal Form (3NF)

> **Rule:** Must satisfy 2NF. No non-key attribute may be **transitively dependent** on the primary key through another non-key attribute.
>
> A transitive dependency exists when: `PK → A → B` (B depends on A, not directly on PK). The fix is always to move `(A, B)` into a new table where `A` becomes the PK.

5.1 USERS

Non-key attributes: `Name, Email, PasswordHash, Role, CreatedAt`

Transitive Dependency Check:

```
UserID → Name          Direct. ✓
UserID → Email         Direct. ✓
UserID → PasswordHash  Direct. ✓
UserID → Role          Direct. ✓
UserID → CreatedAt     Direct. ✓
```

No non-key attribute determines another non-key attribute within this table.

> ✓ **USERS satisfies 3NF.** All attributes are directly and solely determined by `UserID`. No transitive dependencies exist.

5.2 COMPANIES

Non-key attributes: `Name, Industry, Location, ContactEmail, UserID`

Transitive Dependency Check:

```
CompanyID → Name           Direct. ✓
CompanyID → Industry       Direct. ✓
CompanyID → Location       Direct. ✓
CompanyID → ContactEmail   Direct. ✓
CompanyID → UserID         Direct. ✓ (FK, not a transitive dependency — it's a di
```

Could `Industry → Location` be a dependency? Only if every company in one industry is in the same location — this is empirically false and not a business rule of this system. No such functional dependency exists.

> ✓ **COMPANIES satisfies 3NF.** All attributes are directly determined by `CompanyID`. No transitive dependencies exist.

5.3 JOBS

Non-key attributes: `Title, Description, CompanyID, Location, Salary, Deadline, Status`

Transitive Dependency Check:

```
JobID → Title          Direct. ✓
JobID → Description    Direct. ✓
JobID → CompanyID      Direct. ✓  (FK to COMPANIES)
JobID → Location       Direct. ✓  (job's specific work location)
JobID → Salary         Direct. ✓
JobID → Deadline       Direct. ✓
JobID → Status         Direct. ✓
```

Potential concern — `CompanyID → Location`:
One might argue: "The job's location is determined by the company's location, so `JobID → CompanyID → Location` is transitive." However:

- `Jobs.Location` represents the **specific work location for this position** (e.g., a company headquartered in Lahore might post a remote position, or a job in their Karachi office).

- `Companies.Location` represents the company's **registered headquarters address**.
- These are **semantically distinct attributes** — they may coincide but are independently updatable facts.

> ✓ **JOBS satisfies 3NF.** All attributes are directly determined by `JobID`. The `Location` field is position-specific, not transitively derived from `CompanyID → Companies.Location`. This redundancy is addressed in Section 6 as a design recommendation, not a 3NF violation.

5.4 APPLICANTS

Non-key attributes: `UserID, ResumeURL, ExperienceYears, Bio, Gender, University`

Transitive Dependency Check:

```
ApplicantID → UserID          Direct. ✓  (FK)
ApplicantID → ResumeURL       Direct. ✓
ApplicantID → ExperienceYears Direct. ✓
ApplicantID → Bio             Direct. ✓
ApplicantID → Gender          Direct. ✓
ApplicantID → University      Direct. ✓
```

Potential concern — `University → Degree/Level` ?
Only relevant if a `DegreeLevel` or `GPA` attribute were added. With only the university name stored, no further dependency chain exists.

> ✓ **APPLICANTS satisfies 3NF.** All attributes are directly and solely determined by `ApplicantID`.

5.5 SKILLS

Non-key attributes: `SkillName, Category`

Transitive Dependency Check:

```
SkillID → SkillName   Direct. ✓
SkillID → Category    Direct. ✓
```

Potential concern — `Category` as its own entity:
Could `SkillName → Category` be a functional dependency? For example: does knowing the skill name always determine its category? Possibly — "Python" always belongs to

"Programming". However:

- `Category` here is a property of the **skill record**, not a separate entity with its own attributes.
- No other attributes depend on `Category`.
- There is no `CategoryID` or other Category-specific data that would justify a separate table.

Verdict: Even if `SkillName → Category` holds empirically, this does not violate 3NF because `SkillName` is itself a **candidate key** (it has a UNIQUE constraint). A dependency through a candidate key is acceptable in 3NF.

> ✓ **SKILLS satisfies 3NF.** All attributes are directly determined by `SkillID`. `Category` may be considered a property of the skill's canonical name (a candidate key), which is acceptable.

5.6 JOB_SKILLS

Non-key attributes: `RequirementLevel`

Only one non-key attribute — no chain of non-key dependencies is possible.

> ✓ **JOB_SKILLS satisfies 3NF.** With a single non-key attribute, transitive dependencies are structurally impossible.

5.7 APPLICANT_SKILLS

Non-key attributes: `ProficiencyLevel`

Only one non-key attribute — no chain is possible.

> ✓ **APPLICANT_SKILLS satisfies 3NF.** Same reasoning as JOB_SKILLS.

5.8 APPLICATIONS

Non-key attributes: `ApplicantID, JobID, AppliedDate, Status`

Transitive Dependency Check:

```
AppID → ApplicantID   Direct. ✓  (FK)
AppID → JobID         Direct. ✓  (FK)
```

```
AppID → AppliedDate   Direct. ✓
AppID → Status        Direct. ✓
```

Potential concern — `Status` progression:
Could `Status` be determined by interview outcomes? E.g., "If `Interviews.Result = Passed` then `Applications.Status = Hired`." This is **application business logic**, not a stored functional dependency — both fields are independently writable and the relationship is enforced through application code, not schema structure.

> ✓ **APPLICATIONS satisfies 3NF.** All attributes are directly determined by `AppID`. Business logic correlations between Status and Interview Result are not schema-level transitive dependencies.

5.9 INTERVIEWS

Non-key attributes: `AppID, ScheduledDate, Mode, Result`

Transitive Dependency Check:

```
InterviewID → AppID           Direct. ✓  (FK)
InterviewID → ScheduledDate   Direct. ✓
InterviewID → Mode            Direct. ✓
InterviewID → Result          Direct. ✓
```

Could `AppID → ApplicantID → ExperienceYears` be transitive? Only if `ApplicantID` or `ExperienceYears` were stored in this table — they are not. `AppID` is merely a foreign key linking to APPLICATIONS; the applicant's data stays in APPLICANTS.

> ✓ **INTERVIEWS satisfies 3NF.** All attributes are directly determined by `InterviewID`. No applicant or job data is duplicated in this table.

3NF Summary

> **All 9 tables satisfy 3NF.** The key design decisions that prevent transitive dependencies are:
>
> - Applicant profile data (ResumeURL, Bio, Gender, University) is in APPLICANTS, not duplicated in APPLICATIONS or INTERVIEWS.
> - Company data (Name, Industry) is in COMPANIES, not duplicated in JOBS.
> - Skill metadata (SkillName, Category) is in SKILLS, not duplicated in JOB_SKILLS or

> APPLICANT_SKILLS.
>
> - The only cross-table references are via foreign keys — the correct, non-redundant mechanism for representing relationships.

---

6. Schema Refinements & Redundancy Analysis

This section identifies design decisions that, while not strict 3NF violations, represent opportunities for improved clarity, reduced redundancy, or better queryability.

6.1 Location Redundancy: `JOBS.Location` vs `COMPANIES.Location`

Observation:

Both `JOBS` and `COMPANIES` store a `Location` attribute. This could cause confusion:

```
COMPANIES.Location  =  "Lahore, Pakistan"   (headquarters)
JOBS.Location       =  "Remote"             (work arrangement for this position)
JOBS.Location       =  "Karachi Office"     (different from HQ)
```

Is this a 3NF violation?
No. As argued in Section 5.3, these are semantically distinct facts. `Jobs.Location` is not transitively derived from `CompanyID → Companies.Location`.

Recommendation:
Rename the fields to make the distinction unambiguous and avoid developer confusion:

```
-- Before
COMPANIES.Location   VARCHAR   -- ambiguous
JOBS.Location        VARCHAR   -- ambiguous

-- After (recommended)
COMPANIES.HeadquartersLocation   VARCHAR   -- company's registered address
JOBS.WorkLocation                VARCHAR   -- position-specific work location/mod
```

> **Action:** Rename `Companies.Location` → `Companies.HeadquartersLocation` and `Jobs.Location` → `Jobs.WorkLocation` in the schema DDL to eliminate semantic ambiguity. No table restructuring required.

6.2 Skill Category — Potential Lookup Table

Observation:
`Skills.Category` stores values like "Programming", "Data Science", "Soft Skills" as free-text strings. If the same string is typed inconsistently ("Soft Skills" vs "Soft skills" vs "SoftSkills"), category-based filtering queries will fail silently.

Current state:

```
SkillID | SkillName   | Category
--------|-------------|----------
1       | Python      | Programming
2       | SQL         | programming     ← inconsistent capitalization
3       | Leadership  | Soft Skills
```

Recommendation:

Option A — ENUM constraint (simpler, fixed categories):

```
Category ENUM('Programming', 'Data Science', 'Design', 'Management', 'Soft Skills
```

Option B — Separate SKILL_CATEGORIES lookup table (flexible, extensible):

```
SKILL_CATEGORIES (CategoryID PK, CategoryName VARCHAR UNIQUE NOT NULL)
SKILLS           (SkillID PK, SkillName VARCHAR, CategoryID FK → SKILL_CATEGORIES
```

> **Recommendation for this project scope:** Apply an **ENUM constraint** on `Skills.Category` (Option A). It enforces consistency without adding a 10th table to the schema, which is appropriate for a semester-level project. Option B would be preferred in a production system where new skill categories may be added by administrators.

6.3 Role Separation: `USERS.Role` vs Profile Tables

Observation:
A single `USERS` table stores all roles (JobSeeker, Employer, Admin), while role-specific data is stored in COMPANIES (for Employers) and APPLICANTS (for JobSeekers). This is the correct 3NF design — it avoids NULL-heavy "catch-all" tables.

Confirmation that current design is correct:

```
-- Anti-pattern (rejected): One fat table with NULLs
USERS_BAD (UserID, Name, Email, Role,
```

```
            CompanyName, Industry,           ← NULL for JobSeekers
            ResumeURL, ExperienceYears, Bio  ← NULL for Employers)

-- Correct design (current): Separate extension tables
USERS      (UserID, Name, Email, Role, ...)
COMPANIES  (CompanyID, UserID FK, Name, Industry, ...)   ← Employer extension
APPLICANTS (ApplicantID, UserID FK, ResumeURL, Bio, ...)  ← JobSeeker extension
```

> ✓ **No change required.** The current design correctly uses the "Class-Subclass" or "Table-per-Type" pattern, which is the 3NF-compliant approach to role-specific data.

6.4 Application Status vs Interview Result — Coordination

Observation:
`Applications.Status` includes `Hired` and `Rejected` as values. `Interviews.Result` includes `Passed` and `Failed`. These fields overlap in meaning for the final disposition stage.

Current state:

```
Applications.Status  ∈ {Applied, Shortlisted, Rejected, Hired}
Interviews.Result    ∈ {Pending, Passed, Failed}
```

Potential inconsistency:
An `Interviews.Result = 'Passed'` record could coexist with `Applications.Status = 'Rejected'` if the application layer has a bug — both fields can be written independently.

Recommendation:
This is not a normalization issue but a **business rule enforcement** issue. Enforce consistency through:

```
-- Application-layer rule:
-- When Interview.Result is updated to 'Passed',
--   automatically set Applications.Status = 'Hired'
-- When Interview.Result is updated to 'Failed',
--   automatically set Applications.Status = 'Rejected'
```

Or enforce via a `TRIGGER` in MySQL:

```
CREATE TRIGGER sync_application_status
AFTER UPDATE ON Interviews
FOR EACH ROW
```

```
BEGIN
  IF NEW.Result = 'Passed' THEN
    UPDATE Applications SET Status = 'Hired'    WHERE AppID = NEW.AppID;
  ELSEIF NEW.Result = 'Failed' THEN
    UPDATE Applications SET Status = 'Rejected' WHERE AppID = NEW.AppID;
  END IF;
END;
```

> **Recommendation:** Implement a MySQL TRIGGER or application-layer validation to keep `Applications.Status` synchronized with `Interviews.Result`. No schema restructuring required.

6.5 `UNIQUE (ApplicantID, JobID)` — Prevent Duplicate Applications

Observation:
The APPLICATIONS table should enforce that one applicant cannot submit more than one application to the same job. This is a business rule that must be backed by a database constraint.

Implementation:

```
CONSTRAINT uq_application UNIQUE (ApplicantID, JobID)
```

> ✓ **Already identified in schema design.** This constraint eliminates the insertion anomaly of duplicate applications and removes the need for application-layer duplicate checking.

---

7. Final Normalized Schema

The following schema reflects the fully normalized (3NF) design after all refinements.

7.1 USERS

```
USERS (
  UserID       INT          PRIMARY KEY AUTO_INCREMENT,
  Name         VARCHAR(100) NOT NULL,
  Email        VARCHAR(150) NOT NULL UNIQUE,
  PasswordHash VARCHAR(255) NOT NULL,
  Role         ENUM('JobSeeker', 'Employer', 'Admin') NOT NULL,
```

```
  CreatedAt    DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP
)
```

Functional Dependencies:

```
UserID → {Name, Email, PasswordHash, Role, CreatedAt}
```

7.2 COMPANIES

```
COMPANIES (
  CompanyID            INT          PRIMARY KEY AUTO_INCREMENT,
  Name                 VARCHAR(150) NOT NULL,
  Industry             VARCHAR(100) NOT NULL,
  HeadquartersLocation VARCHAR(150) NOT NULL,    -- renamed from Location
  ContactEmail         VARCHAR(150) NOT NULL,
  UserID               INT          NOT NULL,
  FOREIGN KEY (UserID) REFERENCES USERS(UserID)
)
```

Functional Dependencies:

```
CompanyID → {Name, Industry, HeadquartersLocation, ContactEmail, UserID}
```

7.3 JOBS

```
JOBS (
  JobID        INT             PRIMARY KEY AUTO_INCREMENT,
  Title        VARCHAR(200)    NOT NULL,
  Description  TEXT            NOT NULL,
  CompanyID    INT             NOT NULL,
  WorkLocation VARCHAR(150)    NOT NULL,    -- renamed from Location
  Salary       DECIMAL(10, 2)  NULL,
  Deadline     DATE            NOT NULL,
  Status       ENUM('Open', 'Closed', 'Draft') NOT NULL DEFAULT 'Draft',
  FOREIGN KEY (CompanyID) REFERENCES COMPANIES(CompanyID)
)
```

Functional Dependencies:

```
JobID → {Title, Description, CompanyID, WorkLocation, Salary, Deadline, Status}
```

7.4 APPLICANTS

```
APPLICANTS (
  ApplicantID     INT          PRIMARY KEY AUTO_INCREMENT,
  UserID          INT          NOT NULL UNIQUE,
  ResumeURL       VARCHAR(500) NULL,
  ExperienceYears INT          NOT NULL DEFAULT 0,
  Bio             TEXT         NULL,
  Gender          ENUM('Male', 'Female', 'Other', 'Prefer not to say') NULL,
  University      VARCHAR(200) NULL,
  FOREIGN KEY (UserID) REFERENCES USERS(UserID)
)
```

Functional Dependencies:

```
ApplicantID → {UserID, ResumeURL, ExperienceYears, Bio, Gender, University}
```

7.5 SKILLS

```
SKILLS (
  SkillID   INT         PRIMARY KEY AUTO_INCREMENT,
  SkillName VARCHAR(100) NOT NULL UNIQUE,
  Category  ENUM('Programming', 'Data Science', 'Design',
                 'Management', 'Soft Skills', 'DevOps',
                 'Database', 'Networking', 'Other') NOT NULL
)
```

Functional Dependencies:

```
SkillID → {SkillName, Category}
```

7.6 JOB_SKILLS

```
JOB_SKILLS (
  JobSkillID       INT  PRIMARY KEY AUTO_INCREMENT,
```

```
  JobID            INT  NOT NULL,
  SkillID          INT  NOT NULL,
  RequirementLevel ENUM('Beginner', 'Intermediate', 'Expert') NOT NULL,
  FOREIGN KEY (JobID)   REFERENCES JOBS(JobID),
  FOREIGN KEY (SkillID) REFERENCES SKILLS(SkillID),
  CONSTRAINT uq_job_skill UNIQUE (JobID, SkillID)
)
```

Functional Dependencies:

```
(JobID, SkillID) → {RequirementLevel}
```

7.7 APPLICANT_SKILLS

```
APPLICANT_SKILLS (
  AppSkillID      INT  PRIMARY KEY AUTO_INCREMENT,
  ApplicantID     INT  NOT NULL,
  SkillID         INT  NOT NULL,
  ProficiencyLevel ENUM('Beginner', 'Intermediate', 'Expert') NOT NULL,
  FOREIGN KEY (ApplicantID) REFERENCES APPLICANTS(ApplicantID),
  FOREIGN KEY (SkillID)     REFERENCES SKILLS(SkillID),
  CONSTRAINT uq_applicant_skill UNIQUE (ApplicantID, SkillID)
)
```

Functional Dependencies:

```
(ApplicantID, SkillID) → {ProficiencyLevel}
```

7.8 APPLICATIONS

```
APPLICATIONS (
  AppID       INT  PRIMARY KEY AUTO_INCREMENT,
  ApplicantID INT  NOT NULL,
  JobID       INT  NOT NULL,
  AppliedDate DATE NOT NULL DEFAULT (CURRENT_DATE),
  Status      ENUM('Applied', 'Shortlisted', 'Rejected', 'Hired') NOT NULL DEFAUL
  FOREIGN KEY (ApplicantID) REFERENCES APPLICANTS(ApplicantID),
  FOREIGN KEY (JobID)       REFERENCES JOBS(JobID),
  CONSTRAINT uq_application UNIQUE (ApplicantID, JobID)
)
```

Functional Dependencies:

```
AppID           → {ApplicantID, JobID, AppliedDate, Status}
(ApplicantID, JobID) → {AppliedDate, Status}   [candidate key]
```

7.9 INTERVIEWS

```
INTERVIEWS (
  InterviewID   INT      PRIMARY KEY AUTO_INCREMENT,
  AppID         INT      NOT NULL UNIQUE,
  ScheduledDate DATETIME NOT NULL,
  Mode          ENUM('Online', 'Onsite') NOT NULL,
  Result        ENUM('Pending', 'Passed', 'Failed') NOT NULL DEFAULT 'Pending',
  FOREIGN KEY (AppID) REFERENCES APPLICATIONS(AppID)
)
```

Functional Dependencies:

```
InterviewID → {AppID, ScheduledDate, Mode, Result}
AppID       → {ScheduledDate, Mode, Result}   [candidate key — UNIQUE constraint]
```

---

8. Normalization Summary Table

8.1 Normal Form Compliance

Table	1NF	Reason	2NF	Reason	3NF	Reason
USERS	✓	All atomic, single PK	✓	Single PK, no composite	✓	All attrs direct on UserID
COMPANIES	✓	All atomic, single PK	✓	Single PK, no composite	✓	All attrs direct on CompanyID
		All				All attrs

JOBS	✓	atomic, single PK	✓	Single PK, no composite	✓	direct on JobID
APPLICANTS	✓	All atomic, single PK	✓	Single PK, no composite	✓	All attrs direct on ApplicantID
SKILLS	✓	All atomic, single PK	✓	Single PK, no composite	✓	SkillName is candidate key
JOB_SKILLS	✓	Junction table, atomic rows	✓	RequirementLevel fully depends on (JobID, SkillID)	✓	Single non-key attr, no chain
APPLICANT_SKILLS	✓	Junction table, atomic rows	✓	ProficiencyLevel fully depends on (ApplicantID, SkillID)	✓	Single non-key attr, no chain
APPLICATIONS	✓	All atomic, single PK	✓	AppliedDate & Status fully depend on (ApplicantID, JobID)	✓	No non-key → non-key dependency
INTERVIEWS	✓	All atomic, single PK	✓	Single PK, no composite	✓	All attrs direct on InterviewID

8.2 Refinements Applied

Issue	Type	Table(s)	Action Taken
`Location` name ambiguity	Semantic / Naming	JOBS, COMPANIES	Renamed to `WorkLocation` and `HeadquartersLocation`

`Skills.Category` free-text	Data integrity	SKILLS	Applied ENUM constraint on Category
Duplicate application entries	Business rule	APPLICATIONS	Added `UNIQUE (ApplicantID, JobID)`
Duplicate job-skill entries	Data integrity	JOB_SKILLS	Added `UNIQUE (JobID, SkillID)`
Duplicate applicant-skill entries	Data integrity	APPLICANT_SKILLS	Added `UNIQUE (ApplicantID, SkillID)`
Status/Result synchronization	Business rule	APPLICATIONS, INTERVIEWS	Recommended TRIGGER or app-layer enforcement
`USERS.Role` separation	Design pattern	USERS, COMPANIES, APPLICANTS	Confirmed Table-per-Type pattern is correct — no change

---

8.3 Entity Relationship Quick Reference

```
USERS ──────────────── COMPANIES (1:1 optional, via UserID)
USERS ──────────────── APPLICANTS (1:1 optional, via UserID)
COMPANIES ──────────── JOBS (1:N, via CompanyID)
JOBS ──────────────── JOB_SKILLS (1:N, via JobID)
SKILLS ──────────────── JOB_SKILLS (1:N, via SkillID)
APPLICANTS ──────────── APPLICANT_SKILLS (1:N, via ApplicantID)
SKILLS ──────────────── APPLICANT_SKILLS (1:N, via SkillID)
APPLICANTS ──────────── APPLICATIONS (1:N, via ApplicantID)
JOBS ──────────────── APPLICATIONS (1:N, via JobID)
APPLICATIONS ──────── INTERVIEWS (1:1 optional, via AppID)
```

End of NORMALIZATION.md — Job Portal with Intelligent Filtering, Milestone 2
Database Design Specification v2.0 | Aleena Rafiq & Talha Muhammad Bangash